Supplementary material: reproduction protocol and agent prompts

Manuscript SO-26-5308. This file accompanies the revised submission and responds to Reviewer 2, points on repeatability (3.1, 3.2) and release (5). The consensus layer is genuine 3f + 1 Byzantine fault tolerance: a Claude analyzer and a GPT-4o healer propose a fix that is voted on by four independent, model-diverse validators (n = 3f + 1 = 4, quorum = 2f + 1 = 3, f = 1); the analyzer and healer do not vote.

1. Exact model and decoding configuration

Agent	Role	Provider	Model string	Temp	Max tokens	Timeout	Seed
Analyzer	feeds diagnosis (no vote)	Anthropic	claude-sonnet-4-5-20250929	0.3	2048	120 s	n/a (no seed param)
Healer	proposer (no vote)	OpenAI	gpt-4o	0.7	2048	120 s	pinned (OpenAI `seed`)
Validator 1	votes	Anthropic	claude-haiku-4-5-20251001	0.1	512	120 s	n/a (no seed param)
Validator 2	votes	OpenAI	gpt-4o-mini	0.1	512	120 s	pinned (OpenAI `seed`)
Validator 3	votes	Ollama	llama3.1:8b	0.1	512	300 s	pinned (`options.seed`)
Validator 4	votes	Ollama	mistral:7b	0.1	512	300 s	pinned (`options.seed`)

The four validators are model-diverse and cross-provider (two cloud, two local). Independence rule: no validator is the GPT-4o proposer, and the Claude validator is Haiku, distinct from the Sonnet analyzer. `top_p`, `frequency_penalty` and `presence_penalty` are left at provider defaults. The f = 2 tight-bound study (Section 6.13) uses seven homogeneous llama3.1:8b validators (n = 3f + 1 = 7, quorum = 2f + 1 = 5). Sandbox: Docker, network disabled, 512 MB memory, 1 CPU; python:3.11-slim (Python) and eclipse-temurin:17-jdk (Java); isolated-subprocess fallback when Docker is absent. Hardware: HP EliteBook 865 G11, AMD Ryzen 7 8840HS, 64 GB RAM, Windows 11, no GPU (Ollama validators run on the CPU, which is why their per-call timeout is larger).

2. Reproduction commands

```
# environment
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
ollama pull llama3.1:8b
ollama pull mistral:7b
# set ANTHROPIC_API_KEY and OPENAI_API_KEY in .env
```

```
# main consensus runs (genuine 3f+1 heterogeneous panel) vs single-agent baseline
python -m benchmarks --dataset synthetic --pipeline real --sandbox-backend subprocess --mode
consensus --hetero-validators --limit 40 --max-attempts 3 --case-timeout-s 600
python -m benchmarks --dataset synthetic --pipeline real --sandbox-backend subprocess --mode
single-agent --limit 40 --max-attempts 3
python -m benchmarks --dataset bugsinpy --pipeline real --sandbox-backend subprocess --mode
consensus --hetero-validators --bugsinpy-root data/bug_datasets/bugsinpy --bugsinpy-projects
PySnooper ansible --limit 20 --max-attempts 3 --case-timeout-s 600
python -m benchmarks --dataset bugsinpy --pipeline real --sandbox-backend subprocess --mode
single-agent --bugsinpy-root data/bug_datasets/bugsinpy --bugsinpy-projects PySnooper ansible --
limit 20 --max-attempts 3
python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --mode
consensus --hetero-validators --limit 30 --max-attempts 3

# Byzantine fault injection (f=1, one of four independent validators corrupted)
for fault in always_reject always_approve random timeout garbage; do
  python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --mode
consensus --hetero-validators \
    --inject-fault $fault --inject-fault-count 1 --limit 10 --max-attempts 1
done

# f=2 tight bound (n=7, quorum=5; homogeneous llama3.1 replicas): tolerate 2, break at f+1=3
python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --mode
consensus --f 2 --n-validators 7 --inject-fault always_reject --inject-fault-count 2 --limit 5
python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --mode
consensus --f 2 --n-validators 7 --inject-fault always_reject --inject-fault-count 3 --limit 5

# latency n-sweep (independent voters, f=1)
for n in 4 7 10; do
  python -m benchmarks --dataset ecommerce --pipeline real --sandbox-backend subprocess --mode
consensus --f 1 --n-validators $n --limit 5 --max-attempts 1
done

# semantic vs exact quorum study (four heterogeneous Ollama proposers, quorum=3)
python -m benchmarks.semantic_quorum --dataset synthetic --limit 40 --seed 7
python -m benchmarks.semantic_quorum --dataset ecommerce --limit 30 --seed 7

# real Defects4J (Java), after building the defects4j Docker image; real JUnit oracle
python -m benchmarks.defects4j_runner --project Lang --hetero-validators --mode consensus --bugs
1,2,3,4,5,6,7,8,9,10
python -m benchmarks.defects4j_runner --project Math --hetero-validators --mode consensus --bugs
1,2,3,4,5,6,7,8,9,10
```

LLM agents are non-deterministic (Anthropic exposes no seed; cloud providers do not guarantee bit-exact decoding), so an independent rerun will differ in detail. We pin the OpenAI and Ollama seeds and release them; what is invariant across runs is safety — no fix is reported as a repair unless it passes the executable check.

3. Expected headline outcomes (for sanity-checking a rerun)

- **Byzantine tolerance (f = 1):** consensus formation holds across all five fault behaviours with one of four validators corrupted; liveness holds under always-reject and timeout, safety holds under always-approve and random.
- **Tight bound (f = 2, n = 7, quorum = 5):** consensus forms with two Byzantine rejecters and provably fails with three (prepare votes 7 -> 5 -> 4).
- **Repair (BugsInPy, 20 bugs):** consensus repairs 10/20, single-agent 5/20; recall 1.0 against the sandbox oracle, with three genuine split votes (ansible-1, -12, -14; Claude-Haiku dissenting).
- **Defects4J (real JUnit oracle):** two full-suite-verified commons-math repairs (Math-3, Math-5); commons-lang consensus reached on 8/9 evaluated bugs with no full-suite pass; no unverified fix counted as a repair.
- **Quorum rule:** exact-match quorum forms on 2.5-10% of bugs, semantic-equivalence quorum on 70-77% (passing fix in 60%); mean pairwise text agreement 0.18-0.20, behavioural 0.54.
- **Decorrelation:** mean pairwise validator agreement ~1.00 on easy synthetic bugs, 0.925 on real BugsInPy bugs.

4. BugsInPy case mapping (n=20)

Case ID	Project
bip-PySnooper-1	PySnooper
bip-PySnooper-2	PySnooper
bip-PySnooper-3	PySnooper
bip-ansible-1	ansible
bip-ansible-2	ansible
bip-ansible-3	ansible
bip-ansible-4	ansible
bip-ansible-5	ansible
bip-ansible-6	ansible
bip-ansible-7	ansible
bip-ansible-8	ansible
bip-ansible-9	ansible
bip-ansible-10	ansible
bip-ansible-11	ansible
bip-ansible-12	ansible
bip-ansible-13	ansible
bip-ansible-14	ansible
bip-ansible-15	ansible
bip-ansible-16	ansible
bip-ansible-17	ansible

5. Agent prompts (verbatim templates)

5.1 Analyzer (Claude Sonnet, no vote)

```
You are an expert software engineer analyzing a runtime error.
Analyze the following bug and provide a structured analysis.

## Error Information
**Error Message:** <error>

**File:** main.py
**Line:** 3
**Language:** python

## Stack Trace

<trace>


## Code Context

<buggy code>


## Your Task
Provide a JSON response with the following structure:
{
    "bug_type": "<one of: null_pointer, type_error, index_out_of_bounds, division_by_zero,
api_error, timeout, memory_error, syntax_error, logic_error, concurrency_error, unknown>",
    "severity": "<one of: critical, high, medium, low>",
    "root_cause": "<brief explanation of why this error occurred>",
    "affected_code": "<the specific code causing the issue>",
    "suggested_approach": "<how to fix this bug>",
    "related_patterns": ["<list of similar bug patterns>"]
}

Respond ONLY with the JSON object, no additional text.
```

5.2 Healer / proposer (GPT-4o, no vote)

```
You are an expert software engineer fixing a bug in production code.
Generate multiple fix candidates for the following bug.

## Bug Analysis
**Type:** off-by-one
**Severity:** high
**Root Cause:** loop bound
**Suggested Approach:** fix the bound

## Original Code
**File:** main.py
**Line:** 3
**Language:** python
```

<buggy code>

```
## Your Task
Generate 2-3 different fix candidates. For each fix:
1. Provide the complete fixed code
2. Explain what was changed and why
3. Estimate confidence (0.0-1.0)
4. List potential side effects

Respond with a JSON object:
{
    "candidates": [
        {
            "fixed_code": "<complete fixed code>",
            "explanation": "<what was changed and why>",
            "confidence": <0.0-1.0>,
            "changes_description": "<brief description of changes>",
            "potential_side_effects": ["<list of potential issues>"]
        }
    ],
    "recommended_index": <index of best candidate>,
    "generation_strategy": "<brief description of approach>"
}

Important:
- Preserve the original code structure
- Only fix the actual bug, don't refactor
- Ensure the fix is safe for production
- Consider edge cases

Respond ONLY with the JSON object.
```

5.3 Validator (used by all four independent validators)

```
You are reviewing a candidate bug fix as part of a Byzantine consensus stage. The fix has already
been generated by a Healer agent and will be executed in an isolated sandbox after this review.
Your job is structural safety review, NOT exhaustive correctness proof.

## Original Code (buggy)
```

<buggy code>

```
## Proposed Fix
```

<candidate fix>

```
## Fix Explanation
...

## Voting Rules
- Vote IS_VALID = true if the fix plausibly addresses the bug AND introduces no obvious safety
problem.
```

- Vote IS_VALID = false ONLY if you can name a SPECIFIC safety problem (concrete line, condition, or scenario where the fix would crash, corrupt data, or violate an invariant).
- Style preferences, naming choices, missing comments, suboptimal performance, or "could be cleaner" critiques are NOT grounds for rejection. Record them under recommendations.
- If the fix is plausibly correct but you are uncertain, vote IS_VALID = true with moderate confidence (0.5-0.7). The sandbox stage will catch true failures.

Respond with JSON only:

```
{
    "is_valid": <true|false>,
    "confidence": <0.0-1.0>,
    "issues": ["<specific safety problem 1>", "<specific safety problem 2>"],
    "recommendations": ["<style/improvement suggestion 1>"]
}
```

The "issues" list should be EMPTY if the fix is plausibly safe. Only populate it with concrete, specific problems.